

-  CHEAT SHEET THỐNG KÊ MÔ TẢ - TỔNG HỢP TOÀN TẬP
 -  QUY TẮC VÀNG: NHÌN ĐỀ → TÌM MÃ → COPY → THAY TÊN → CHẠY!
 -  IMPORT TOÀN BỘ THƯ VIỆN CẦN THIẾT
 -  MENU TÌM KIẾM SIÊU TỐC
 -  THẤY TỪ KHÓA GÌ TRONG ĐỀ → CTRL+F TÌM MÃ ĐÓ
 -  KHU VỰC COPY CODE
 - A01 - ĐỌC FILE + HIỂN THỊ
 - A02 - SỐ DÒNG CỘT + THÔNG TIN CƠ BẢN
 - A03 - THIẾU DỮ LIỆU
 - A04 - PHÂN LOẠI THUỘC TÍNH
 - B01 - ĐẾM SỐ LƯỢNG
 - B02 - TÌM NHIỀU NHẤT/ÍT NHẤT
 - B03 - TỶ LỆ PHẦN TRĂM
 - B04 - SO SÁNH THEO NHÓM
 - C01 - TRUNG BÌNH, TRUNG VỊ
 - C02 - ĐỘ LỆCH CHUẨN, PHƯƠNG SAI
 - C03 - PHÂN VỊ
 - D01 - BIỂU ĐỒ CỘT
 - D02 - BIỂU ĐỒ TRÒN
 - D03 - HISTOGRAM
 - D04 - BOXPLOT ĐƠN GIẢN
 - D05 - BOXPLOT SO SÁNH NHÓM
 - D06 - MA TRẬN TƯƠNG QUAN
 - D07 - SCATTER PLOT
 - E01 - XÁC SUẤT NHỊ THỨC
 - E02 - XÁC SUẤT POISSON
 - E03 - XÁC SUẤT CHUẨN
 - F01 - LẤY MẪU
 - F02 - MÔ PHỎNG
 - G01 - ĐỔI ĐƠN VỊ
 - G02 - TÍNH BMI VÀ PHÂN LOẠI
 - G03 - XỬ LÝ DATETIME
 - H01 - HISTOGRAM NHIỀU CỘT
 - H02 - SUBPLOT NHIỀU BIỂU ĐỒ
 - H03 - PHÂN NHÓM THEO NGƯỜI
 -  TEMPLATE ĐỀ HOÀN CHỈNH

- TEMPLATE 1: PHÂN TÍCH DỮ LIỆU TỔNG QUÁT
- TEMPLATE 2: PHÂN PHỐI XÁC SUẤT
- TEMPLATE 3: TIPS DATA ANALYSIS
- 🚨 CHECKLIST THI QUAN TRỌNG
 - ✅ TRƯỚC KHI BẮT ĐẦU:
 - ✅ KHI LÀM BÀI:
 - ✅ TRƯỚC KHI NỘP:
 - 🔧 SỬA LỖI NHANH:
 - 📊 CHIẾN THUẬT THI:
 - 💡 MẸO HAY:
- 🎨 CÁC LOẠI BIỂU ĐỒ VÀ KHI NÀO DÙNG

🔥 CHEAT SHEET THỐNG KÊ MÔ TẢ - TỔNG HỢP TOÀN TẬP

🎯 QUY TẮC VÀNG: NHÌN ĐỀ → TÌM MÃ
→ COPY → THAY TÊN → CHẠY!

📖 IMPORT TOÀN BỘ THƯ VIỆN CẦN THIẾT

```
# Import cơ bản
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# Import phân phối xác suất
from scipy.stats import binom, poisson, norm, t

# Thiết lập hiển thị
plt.style.use("ggplot")
plt.rcParams['figure.figsize'] = (10, 6)
```



MENU TÌM KIẾM SIÊU TỐC



THẤY TỪ KHÓA GÌ TRONG ĐỀ → CTRL+F
TÌM MÃ ĐÓ

THẤY TRONG ĐỀ	TÌM MÃ	THAY GÌ
đọc dữ liệu, hiển thị	A01	tên file
bao nhiêu dòng, bao nhiêu cột	A02	không thay
thiếu dữ liệu, missing, null	A03	không thay
định tính, định lượng, phân loại	A04	không thay
có bao nhiêu, value_counts	B01	tên cột
nhiều nhất, ít nhất, top	B02	tên cột
tỷ lệ, %, phần trăm	B03	tên cột
theo nhóm, groupby, so sánh	B04	2 tên cột
trung bình, mean, median	C01	tên cột
độ lệch chuẩn, std, var	C02	tên cột
phân vị, quantile, Q1, Q3	C03	tên cột
biểu đồ cột, bar chart	D01	tên cột
biểu đồ tròn, pie chart	D02	tên cột
histogram, phân bố	D03	tên cột
boxplot, hộp râu	D04	tên cột
so sánh boxplot, theo hãg	D05	2 tên cột
ma trận tương quan, heatmap	D06	không thay
scatter plot, tương quan	D07	2 tên cột
xác suất, P(, nhị thức	E01	n, p, k
Poisson, λ , lambda	E02	λ , k
chuẩn, normal, mm, đường kính	E03	μ , σ , x

THẤY TRONG ĐỀ	TÌM MÃ	THAY GÌ
lấy mẫu, sample	F01	số mẫu
mô phỏng, simulation, 100 lần	F02	số lần
đổi đơn vị, inches, pounds	G01	công thức
BMI, kg/m ² , béo phì	G02	height, weight
datetime, thời gian, tháng	G03	cột time
histogram nhiều cột	H01	danh sách cột
subplot, nhiều biểu đồ	H02	số dòng, cột
phân nhóm theo ngưỡng	H03	ngưỡng phân nhóm



KHU VỰC COPY CODE

A01 - ĐỌC FILE + HIỂN THỊ

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# ★ THAY TÊN FILE
df = pd.read_csv('TÊN_FILE.csv')
print("5/7/10 dòng đầu:")
print(df.head(5)) # Thay 5→7→10 tùy đề
print("5/7/10 dòng cuối:")
print(df.tail(5))
print("Dữ liệu ngẫu nhiên:")
print(df.sample(5))
```

A02 - SỐ DÒNG CỘT + THÔNG TIN CƠ BẢN

```
print(f"Dữ liệu có {df.shape[0]} dòng và {df.shape[1]} cột")
print(f"Kích thước: {df.shape}")
print("\nThông tin cột:")
print(df.info())
print("\nThống kê mô tả:")
```

```
print(df.describe())
print("\nTên các cột:")
print(df.columns.tolist())
```

A03 - THIẾU DỮ LIỆU

```
print("Số giá trị thiếu:")
print(df.isnull().sum())
print(f"Tổng thiếu: {df.isnull().sum().sum()}")

# Hiển thị các dòng thiếu dữ liệu
print("Các dòng có giá trị thiếu:")
print(df[df.isnull().any(axis=1)])

# Bỏ dòng thiếu dữ liệu
df_clean = df.dropna()
print(f"Sau khi bỏ thiếu: {df_clean.shape[0]} dòng")

# Điền giá trị thiếu
df['CỘT'] = df['CỘT'].fillna('Unknown') # Cho cột text
df = df.fillna(0) # Điền 0 cho tất cả
```

A04 - PHÂN LOẠI THUỘC TÍNH

```
print("Thuộc tính định lượng (số):", df.select_dtypes(include=[np.number]).columns.tolist())
print("Thuộc tính định tính (chữ):", df.select_dtypes(include=['object']).columns.tolist())

# Hiển thị giá trị phân biệt cho thuộc tính định tính
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    print(f"\n{col}: {df[col].unique()}")
    print(f"Số giá trị phân biệt: {df[col].nunique()}")
```

B01 - ĐẾM SỐ LƯỢNG

```
# ★ THAY TÊN CỘT
counts = df['TÊN_CỘT'].value_counts()
print("Số lượng từng loại:")
print(counts)
```

```
print(f"Tổng có {df['TÊN_CỘT'].nunique()} loại khác nhau")

# Thống kê nâng cao
print("\nBảng phân phối tần số:")
freq_table = df['TÊN_CỘT'].value_counts().sort_index()
print(freq_table)
```

B02 - TÌM NHIỀU NHẤT/ÍT NHẤT

```
# ★ THAY TÊN CỘT
top = df['TÊN_CỘT'].value_counts().head(1)
print(f"Nhiều nhất: {top.index[0]} ({top.iloc[0]} lần)")

least = df['TÊN_CỘT'].value_counts().tail(1)
print(f"Ít nhất: {least.index[0]} ({least.iloc[0]} lần)")

# Mode (giá trị xuất hiện nhiều nhất)
mode_value = df['TÊN_CỘT'].mode().iloc[0]
print(f"Mode: {mode_value}")

# Top 10
print("Top 10 giá trị:")
top_10 = df['TÊN_CỘT'].value_counts().head(10)
print(top_10)
```

B03 - TỶ LỆ PHẦN TRĂM

```
# ★ THAY TÊN CỘT
counts = df['TÊN_CỘT'].value_counts()
percentages = (counts / len(df) * 100).round(1)
print("Tỷ lệ %:")
for i, value in enumerate(counts.index):
    print(f"{value}: {counts.iloc[i]} ({percentages.iloc[i]}%)")

# Tỷ lệ trực tiếp
percent_direct = df['TÊN_CỘT'].value_counts(normalize=True) * 100
print("\nTỷ lệ % trực tiếp:")
print(percent_direct.round(1))
```

B04 - SO SÁNH THEO NHÓM

```
# ★ THAY 2 TÊN CỘT
# Bảng chéo 2 biến
crosstab = pd.crosstab(df['CỘT_NHÓM_1'], df['CỘT_NHÓM_2'])
print("Bảng chéo:")
print(crosstab)

# Tỷ lệ theo nhóm
crosstab_percent = pd.crosstab(df['CỘT_NHÓM_1'], df['CỘT_NHÓM_2'],
normalize='index') * 100
print("Tỷ lệ % theo hàng:")
print(crosstab_percent.round(1))

# Group by cơ bản
grouped = df.groupby('CỘT_NHÓM')['CỘT_GIÁ_TRỊ'].agg(['count', 'mean',
'std']).round(2)
print("Thống kê theo nhóm:")
print(grouped)
```

C01 - TRUNG BÌNH, TRUNG VỊ

```
# ★ THAY TÊN CỘT
print(f"Trung bình: {df['TÊN_CỘT'].mean():.3f}")
print(f"Trung vị: {df['TÊN_CỘT'].median():.3f}")
print(f"Mode: {df['TÊN_CỘT'].mode().iloc[0] if len(df['TÊN_CỘT'].mode()) > 0 else
'N/A'}")

# Thống kê tổng hợp
print("\nThống kê tổng hợp:")
print(df['TÊN_CỘT'].describe())

# Theo nhóm
group_stats = df.groupby('CỘT_NHÓM')['CỘT_GIÁ_TRỊ'].agg(['mean', 'median', 'std',
'count']).round(2)
print("Thống kê theo nhóm:")
print(group_stats)
```

C02 - ĐỘ LỆCH CHUẨN, PHƯƠNG SAI

```
# ★ THAY TÊN CỘT
print(f"Độ lệch chuẩn: {df['TÊN_CỘT'].std():.3f}")
print(f"Phương sai: {df['TÊN_CỘT'].var():.3f}")
print(f"Min: {df['TÊN_CỘT'].min():.3f}")
print(f"Max: {df['TÊN_CỘT'].max():.3f}")
print(f"Khoảng (Range): {df['TÊN_CỘT'].max() - df['TÊN_CỘT'].min():.3f}")
```

```
# Hệ số biến thiên
cv = (df['TÊN_CỘT'].std() / df['TÊN_CỘT'].mean()) * 100
print(f"Hệ số biến thiên: {cv:.2f}%")
```

C03 - PHÂN VỊ

```
# ★ THAY TÊN CỘT
q1 = df['TÊN_CỘT'].quantile(0.25)
q2 = df['TÊN_CỘT'].quantile(0.50) # Trung vị
q3 = df['TÊN_CỘT'].quantile(0.75)

print(f"Q1 (25%): {q1:.3f}")
print(f"Q2 (50%): {q2:.3f}")
print(f"Q3 (75%): {q3:.3f}")
print(f"IQR: {q3-q1:.3f}")

# Các phân vị khác
print(f"P10: {df['TÊN_CỘT'].quantile(0.10):.3f}")
print(f"P90: {df['TÊN_CỘT'].quantile(0.90):.3f}")

# Miền giá trị và miền phân vị
print(f"Miền giá trị: [{df['TÊN_CỘT'].min():.1f}, {df['TÊN_CỘT'].max():.1f}]")
print(f"Miền phân vị (Q1, Q3): [{q1:.1f}, {q3:.1f}]")
```

D01 - BIỂU ĐỒ CỘT

```
# ★ THAY TÊN CỘT
plt.figure(figsize=(10, 6))
counts = df['TÊN_CỘT'].value_counts()
counts.plot(kind='bar', color='skyblue')
plt.title('Biểu đồ cột - TÊN_CỘT')
plt.xlabel('Danh mục')
plt.ylabel('Số lượng')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# So sánh theo nhóm
plt.figure(figsize=(12, 6))
df.groupby('CỘT_NHÓM')['CỘT_GIÁ_TRỊ'].mean().plot(kind='bar', color='lightcoral')
plt.title('So sánh trung bình theo nhóm')
plt.ylabel('Giá trị trung bình')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
# Bar chart với seaborn
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='TÊN_CỘT', palette='viridis')
plt.title('Biểu đồ cột với Seaborn')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

D02 - BIỂU ĐỒ TRÒN

```
# ★ THAY TÊN CỘT
plt.figure(figsize=(10, 8))
counts = df['TÊN_CỘT'].value_counts()
plt.pie(counts.values, labels=counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Biểu đồ tròn - TÊN_CỘT')
plt.axis('equal')
plt.tight_layout()
plt.show()

# Pie chart với explode
plt.figure(figsize=(10, 8))
explode = [0.05] * len(counts) # Tách nhỏ các phần
plt.pie(counts.values, labels=counts.index, autopct='%1.1f%%',
        explode=explode, startangle=90, shadow=True)
plt.title('Biểu đồ tròn có hiệu ứng')
plt.axis('equal')
plt.show()
```

D03 - HISTOGRAM

```
# ★ THAY TÊN CỘT
plt.figure(figsize=(10, 6))
plt.hist(df['TÊN_CỘT'].dropna(), bins=30, color='lightgreen', alpha=0.7,
edgecolor='black')
plt.title('Histogram - TÊN_CỘT')
plt.xlabel('Giá trị')
plt.ylabel('Tần suất')
plt.grid(axis='y', alpha=0.3)

# Thêm đường trung bình
mean_val = df['TÊN_CỘT'].mean()
plt.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'TB:
{mean_val:.2f}')
plt.legend()
plt.show()

# Histogram nhiều cột cùng lúc
```

```
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
cols = ['CỘT_1', 'CỘT_2', 'CỘT_3']
for i, col in enumerate(cols):
    axes[i].hist(df[col].dropna(), bins=20, alpha=0.7)
    axes[i].set_title(f'Histogram {col}')
    axes[i].set_xlabel(col)
plt.tight_layout()
plt.show()
```

D04 - BOXPLOT ĐƠN GIẢN

```
# ★ THAY TÊN CỘT
plt.figure(figsize=(8, 6))
plt.boxplot(df['TÊN_CỘT'].dropna())
plt.title('Boxplot - TÊN_CỘT')
plt.ylabel('Giá trị')
plt.show()

# Nhiều cột cùng lúc với pandas
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
cols = ['CỘT_1', 'CỘT_2', 'CỘT_3']
for i, col in enumerate(cols):
    axes[i].boxplot(df[col].dropna())
    axes[i].set_title(f'Boxplot {col}')
    axes[i].set_ylabel(col)
plt.tight_layout()
plt.show()

# Boxplot với pandas
df[['CỘT_1', 'CỘT_2', 'CỘT_3']].boxplot(figsize=(10, 6))
plt.title('Boxplot nhiều cột')
plt.show()
```

D05 - BOXPLOT SO SÁNH NHÓM

```
# ★ THAY 2 TÊN CỘT
plt.figure(figsize=(12, 8))

# Cách 1: Pandas
df.boxplot(column='CỘT_GIÁ_TRỊ', by='CỘT_NHÓM', figsize=(12, 8))
plt.title('Boxplot so sánh theo nhóm')
plt.suptitle('') # Xóa title mặc định
plt.show()

# Cách 2: Seaborn (đẹp hơn)
plt.figure(figsize=(12, 8))
sns.boxplot(data=df, x='CỘT_NHÓM', y='CỘT_GIÁ_TRỊ')
```

```
plt.title('Boxplot so sánh theo nhóm')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Boxplot theo nhiều nhóm
plt.figure(figsize=(12, 8))
sns.boxplot(data=df, x='CỘT_NHÓM_1', y='CỘT_GIÁ_TRỊ', hue='CỘT_NHÓM_2')
plt.title('Boxplot so sánh theo 2 nhóm')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

D06 - MA TRẬN TƯƠNG QUAN

```
# Ma trận tương quan cho các cột số
numeric_cols = df.select_dtypes(include=[np.number]).columns
correlation_matrix = df[numeric_cols].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0,
            square=True, linewidths=0.5, fmt='.2f')
plt.title('Ma trận tương quan')
plt.tight_layout()
plt.show()

# In ma trận tương quan
print("Ma trận tương quan:")
print(correlation_matrix.round(3))

# Tìm tương quan cao nhất
high_corr = correlation_matrix.abs().unstack().sort_values(ascending=False)
high_corr = high_corr[high_corr < 1] # Bỏ tương quan với chính nó
print(f"\nTương quan cao nhất: {high_corr.head(5)}")
```

D07 - SCATTER PLOT

```
# ★ THAY 2 TÊN CỘT
plt.figure(figsize=(10, 6))
plt.scatter(df['CỘT_X'], df['CỘT_Y'], alpha=0.6)
plt.title('Tương quan giữa X và Y')
plt.xlabel('CỘT_X')
plt.ylabel('CỘT_Y')
plt.grid(True, alpha=0.3)

# Thêm đường xu hướng
z = np.polyfit(df['CỘT_X'].dropna(), df['CỘT_Y'].dropna(), 1)
```

```

p = np.poly1d(z)
plt.plot(df['CỘT_X'], p(df['CỘT_X']), "r--", alpha=0.8, linewidth=2)

# Tính hệ số tương quan
correlation = df['CỘT_X'].corr(df['CỘT_Y'])
plt.title(f'Tương quan: {correlation:.3f}')
plt.show()

# Scatter plot với seaborn
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='CỘT_X', y='CỘT_Y', hue='CỘT_NHÓM')
plt.title('Scatter plot theo nhóm')
plt.show()

```

E01 - XÁC SUẤT NHỊ THỨC

```

from scipy.stats import binom

# ★ THAY SỐ
n = 10      # Số lần thử
p = 0.4     # Xác suất thành công

# Tạo phân phối
binomial_dist = binom(n, p)

# a.  $P(X = k)$ 
k = 4
prob_k = binomial_dist.pmf(k)
print(f"P(X = {k}) = {prob_k:.6f}")

# b.  $P(X \leq k)$ 
prob_leq = binomial_dist.cdf(k)
print(f"P(X <= {k}) = {prob_leq:.6f}")

# c.  $P(X > k)$ 
prob_gt = 1 - binomial_dist.cdf(k)
print(f"P(X > {k}) = {prob_gt:.6f}")

# d.  $E(X)$  và  $Var(X)$ 
print(f"E(X) = {binomial_dist.mean()}")
print(f"Var(X) = {binomial_dist.var()}")

# e. Vẽ đồ thị PMF
x_values = range(0, n+1)
probs = [binomial_dist.pmf(x) for x in x_values]
plt.figure(figsize=(10, 6))
plt.bar(x_values, probs, alpha=0.7, color='lightblue')
plt.title('Phân phối xác suất nhị thức')
plt.xlabel('Số lần thành công')
plt.ylabel('P(X = k)')
plt.show()

```

```
# f. Vẽ đồ thị CDF
cdf_values = [binomial_dist.cdf(x) for x in x_values]
plt.figure(figsize=(10, 6))
plt.plot(x_values, cdf_values, 'bo-', markersize=8)
plt.title('Hàm phân phối tích lũy')
plt.xlabel('k')
plt.ylabel('P(X ≤ k)')
plt.grid(True)
plt.show()
```

E02 - XÁC SUẤT POISSON

```
from scipy.stats import poisson

# ★ THAY SỐ
lam = 3 # Lambda (trung bình)

# Tạo phân phối
poisson_dist = poisson(lam)

# a. P(X = k)
k = 2
prob_k = poisson_dist.pmf(k)
print(f"P(X = {k}) = {prob_k:.6f}")

# b. P(X ≤ k)
prob_leq = poisson_dist.cdf(k)
print(f"P(X ≤ {k}) = {prob_leq:.6f}")

# c. P(X > k)
prob_gt = 1 - poisson_dist.cdf(k)
print(f"P(X > {k}) = {prob_gt:.6f}")

# d. E(X) và Var(X)
print(f"E(X) = {poisson_dist.mean()}")
print(f"Var(X) = {poisson_dist.var()}")

# e. Vẽ đồ thị PMF
x_values = range(0, lam*3+1)
probs = [poisson_dist.pmf(x) for x in x_values]
plt.figure(figsize=(10, 6))
plt.bar(x_values, probs, alpha=0.7, color='lightcoral')
plt.title('Phân phối Poisson')
plt.xlabel('k')
plt.ylabel('P(X = k)')
plt.show()

# Ví dụ ứng dụng thời gian
print("\nÁp dụng cho thời gian:")
lam_hour = 10 # 10 sự kiện/giờ
print(f"P(5 sự kiện trong 1h) = {poisson.pmf(5, lam_hour):.6f}")
```

```
print(f"P(15 sự kiện trong 2h) = {poisson.pmf(15, lam_hour*2):.6f}")
print(f"P(5 sự kiện trong 30p) = {poisson.pmf(5, lam_hour*0.5):.6f}")
```

E03 - XÁC SUẤT CHUẨN

```
from scipy.stats import norm

# ★ THAY SỐ
mu = 500      # Trung bình
sigma = 4     # Độ lệch chuẩn

# Tạo phân phối
normal_dist = norm(mu, sigma)

# a.  $P(X < a)$ 
a = 505
prob_less = normal_dist.cdf(a)
print(f"P(X < {a}) = {prob_less:.6f}")

# b.  $P(a < X < b)$ 
b = 510
prob_between = normal_dist.cdf(b) - normal_dist.cdf(a)
print(f"P({a} < X < {b}) = {prob_between:.6f}")

# c.  $P(X > a)$ 
prob_greater = 1 - normal_dist.cdf(a)
print(f"P(X > {a}) = {prob_greater:.6f}")

# d. Tìm giá trị tại phân vị
percentile = 0.95
value_at_percentile = normal_dist.ppf(percentile)
print(f"Giá trị tại phân vị {percentile*100}%: {value_at_percentile:.2f}")

# e. Vẽ đồ thị PDF
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
y = normal_dist.pdf(x)
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'b-', linewidth=2, label=f' $\mu={mu}$ ,  $\sigma={sigma}$ ')
plt.title('Phân phối chuẩn')
plt.xlabel('Giá trị')
plt.ylabel('Mật độ xác suất')
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()

# f. Phân phối chuẩn tắc
print(f"\nPhân phối chuẩn tắc:")
z = (a - mu) / sigma
print(f"Z-score của {a}: {z:.2f}")
print(f"P(Z < {z:.2f}) = {norm.cdf(z):.6f}")
```

F01 - LẤY MẪU

```
# ★ THAY SỐ MẪU
sample_size = 50
sample_df = df.sample(n=sample_size, random_state=42)

print(f"Mẫu gốc: {len(df)} dòng")
print(f"Mẫu lấy: {len(sample_df)} dòng")

# So sánh thống kê
original_mean = df['CỘT_SỐ'].mean()
sample_mean = sample_df['CỘT_SỐ'].mean()
error = abs(sample_mean - original_mean)

print(f"Trung bình gốc: {original_mean:.3f}")
print(f"Trung bình mẫu: {sample_mean:.3f}")
print(f"Sai số: {error:.3f}")

# Lấy mẫu theo tỷ lệ
sample_percent = df.sample(frac=0.1, random_state=42) # 10%
print(f"Mẫu 10%: {len(sample_percent)} dòng")
```

F02 - MÔ PHỎNG

```
# ★ THAY SỐ LẦN VÀ SỐ MẪU
n_simulations = 100
sample_size = 50
results = []

np.random.seed(42) # Để kết quả ổn định
for i in range(n_simulations):
    sample = df.sample(n=sample_size)
    # Tính thống kê cần thiết (thay đổi tùy đề)
    result = sample['CỘT_SỐ'].mean() # ★ THAY
    results.append(result)

# Thống kê kết quả mô phỏng
print(f"Trung bình {n_simulations} lần mô phỏng: {np.mean(results):.3f}")
print(f"Độ lệch chuẩn: {np.std(results):.3f}")

# Vẽ histogram kết quả
plt.figure(figsize=(10, 6))
plt.hist(results, bins=20, alpha=0.7, edgecolor='black', color='lightblue')
plt.title(f'Histogram {n_simulations} lần mô phỏng (n={sample_size})')
plt.xlabel('Kết quả mẫu')
plt.ylabel('Tần suất')
plt.axvline(np.mean(results), color='red', linestyle='--',
            label=f'TB: {np.mean(results):.3f}')
plt.legend()
```

```
plt.grid(axis='y', alpha=0.3)
plt.show()

# So sánh các cỡ mẫu khác nhau
sample_sizes = [10, 30, 50, 100]
fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for i, size in enumerate(sample_sizes):
    results_size = []
    for j in range(100):
        sample = df.sample(n=size)
        result = sample['CỘT_SỐ'].mean()
        results_size.append(result)

    axes[i].hist(results_size, bins=15, alpha=0.7)
    axes[i].set_title(f'n={size}, SD={np.std(results_size):.3f}')
    axes[i].axvline(np.mean(results_size), color='red', linestyle='--')

plt.tight_layout()
plt.show()
```

G01 - ĐỔI ĐƠN VỊ

```
# ★ THAY CÔNG THỨC ĐỔI ĐƠN VỊ
# Inches sang cm
df['height_cm'] = df['height'] * 2.54
print(f"Chiều cao TB: {df['height_cm'].mean():.1f} cm")

# Pounds sang kg
df['weight_kg'] = df['weight'] * 0.453592
print(f"Cân nặng TB: {df['weight_kg'].mean():.1f} kg")

# Fahrenheit sang Celsius
df['temp_celsius'] = (df['temp_fahrenheit'] - 32) * 5/9

# Feet sang mét
df['distance_m'] = df['distance_ft'] * 0.3048

# Gallon sang lít
df['volume_l'] = df['volume_gallon'] * 3.78541

print("Đã hoàn thành chuyển đổi đơn vị!")
```

G02 - TÍNH BMI VÀ PHÂN LOẠI


```

# Tính BMI (cần có height_cm và weight_kg)
df['BMI'] = df['weight_kg'] / (df['height_cm'] / 100) ** 2

# Phân loại BMI
def classify_bmi(bmi):
    if pd.isna(bmi):
        return 'Unknown'
    elif bmi < 18.5:
        return 'Thiếu cân'
    elif bmi < 25:
        return 'Bình thường'
    elif bmi < 30:
        return 'Thừa cân'
    else:
        return 'Béo phì'

df['BMI_category'] = df['BMI'].apply(classify_bmi)

# Thống kê BMI
print("BMI trung bình theo giới tính:")
bmi_by_gender = df.groupby('gender')['BMI'].agg(['mean', 'std']).round(2)
print(bmi_by_gender)

# Tỷ lệ phân loại BMI
obesity_stats = df['BMI_category'].value_counts()
obesity_percent = (obesity_stats / len(df) * 100).round(1)
print("\nTỷ lệ phân loại BMI:")
for category in obesity_stats.index:
    print(f"{category}: {obesity_stats[category]} ({obesity_percent[category]}%)")

# Tỷ lệ béo phì theo nhóm tuổi
df['age_group'] = pd.cut(df['age'], bins=[0, 30, 45, 60, 100],
                        labels=['<30', '30-44', '45-59', '60+'])
obesity_by_age = pd.crosstab(df['age_group'], df['BMI_category'],
                             normalize='index') * 100
print("\nTỷ lệ % BMI theo nhóm tuổi:")
print(obesity_by_age.round(1))

```

G03 - XỬ LÝ DATETIME

```

# ★ THAY TÊN CỘT THỜI GIAN
# Chuyển cột thời gian thành datetime
df['time'] = pd.to_datetime(df['time'])

# Trích xuất thông tin thời gian
df['year'] = df['time'].dt.year
df['month'] = df['time'].dt.month
df['day'] = df['time'].dt.day
df['weekday'] = df['time'].dt.day_name()
df['hour'] = df['time'].dt.hour
df['quarter'] = df['time'].dt.quarter

```

```

# Thống kê theo thời gian
monthly_stats = df.groupby('month').size()
print("Số lượng theo tháng:")
print(monthly_stats)

# Vẽ biểu đồ theo thời gian
plt.figure(figsize=(12, 6))
monthly_stats.plot(kind='bar', color='lightblue')
plt.title('Thống kê theo tháng')
plt.xlabel('Tháng')
plt.ylabel('Số lượng')
plt.xticks(rotation=0)
plt.grid(axis='y', alpha=0.3)
plt.show()

# Thống kê theo ngày trong tuần
weekday_stats = df['weekday'].value_counts()
print("\nThống kê theo ngày trong tuần:")
print(weekday_stats)

# Tìm xu hướng theo thời gian
time_trend = df.groupby('time').size().resample('M').sum()
plt.figure(figsize=(12, 6))
time_trend.plot()
plt.title('Xu hướng theo thời gian')
plt.ylabel('Số lượng')
plt.show()

```

H01 - HISTOGRAM NHIỀU CỘT

```

# ★ THAY DANH SÁCH CỘT
columns = ['CỘT_1', 'CỘT_2', 'CỘT_3', 'CỘT_4']

# Cách 1: Subplot
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
axes = axes.flatten()

for i, col in enumerate(columns):
    axes[i].hist(df[col].dropna(), bins=20, alpha=0.7, color=f'C{i}')
    axes[i].set_title(f'Histogram {col}')
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Tần suất')
    axes[i].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

# Cách 2: Pandas hist
df[columns].hist(figsize=(15, 10), bins=20)
plt.suptitle('Histogram tất cả các cột', fontsize=16)

```

```

plt.tight_layout()
plt.show()

# Cách 3: Chồng histogram
plt.figure(figsize=(12, 6))
for col in columns:
    plt.hist(df[col].dropna(), bins=20, alpha=0.5, label=col)
plt.title('Histogram chồng')
plt.xlabel('Giá trị')
plt.ylabel('Tần suất')
plt.legend()
plt.show()

print("Nhận xét phân phối:")
for col in columns:
    skew = df[col].skew()
    if skew > 0.5:
        dist_type = "lệch phải"
    elif skew < -0.5:
        dist_type = "lệch trái"
    else:
        dist_type = "gần đối xứng"
    print(f"- {col}: {dist_type} (skewness: {skew:.2f})")

```

H02 - SUBPLOT NHIỀU BIỂU ĐỒ

```

# ★ THAY SỐ DÒNG, CỘT VÀ LOẠI BIỂU ĐỒ
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# Histogram
axes[0, 0].hist(df['CỘT_SỐ'].dropna(), bins=20, alpha=0.7)
axes[0, 0].set_title('Histogram')

# Boxplot
axes[0, 1].boxplot(df['CỘT_SỐ'].dropna())
axes[0, 1].set_title('Boxplot')

# Scatter plot
axes[0, 2].scatter(df['CỘT_X'], df['CỘT_Y'], alpha=0.6)
axes[0, 2].set_title('Scatter Plot')

# Bar chart
counts = df['CỘT_PHÂN_LOẠI'].value_counts()
axes[1, 0].bar(counts.index, counts.values)
axes[1, 0].set_title('Bar Chart')
axes[1, 0].tick_params(axis='x', rotation=45)

# Line plot
time_data = df.groupby('THỜI_GIAN')['GIÁ_TRỊ'].mean()
axes[1, 1].plot(time_data.index, time_data.values, marker='o')
axes[1, 1].set_title('Line Plot')

```

```
# Pie chart
pie_data = df['CỘT_PHÂN_LOẠI'].value_counts()
axes[1, 2].pie(pie_data.values, labels=pie_data.index, autopct='%1.1f%%')
axes[1, 2].set_title('Pie Chart')

plt.tight_layout()
plt.show()
```

H03 - PHÂN NHÓM THEO NGƯỠNG

```
# ★ THAY NGƯỠNG PHÂN NHÓM
# Phân nhóm theo khoảng giá trị
df['nhom_tuoi'] = pd.cut(df['age'],
                        bins=[0, 25, 35, 50, 100],
                        labels=['Trẻ', 'Trung niên', 'Lớn tuổi', 'Cao tuổi'])

# Phân nhóm theo phân vị
df['nhom_thu_nhap'] = pd.qcut(df['income'],
                             q=4,
                             labels=['Thấp', 'Trung bình', 'Cao', 'Rất cao'])

# Phân nhóm theo điều kiện
def phan_loai_bmi(bmi):
    if bmi < 18.5:
        return 'Gầy'
    elif bmi < 25:
        return 'Bình thường'
    elif bmi < 30:
        return 'Thừa cân'
    else:
        return 'Béo phì'

df['nhom_bmi'] = df['BMI'].apply(phan_loai_bmi)

# Thống kê theo nhóm
print("Thống kê theo nhóm tuổi:")
print(df['nhom_tuoi'].value_counts())

print("\nThống kê theo nhóm thu nhập:")
print(df['nhom_thu_nhap'].value_counts())

# So sánh giữa các nhóm
comparison = df.groupby('nhom_tuoi')['income'].agg(['mean', 'median',
'std']).round(2)
print("\nSo sánh thu nhập theo nhóm tuổi:")
print(comparison)
```



TEMPLATE ĐỀ HOÀN CHỈNH

TEMPLATE 1: PHÂN TÍCH DỮ LIỆU TỔNG QUÁT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

# 1. ĐỌC VÀ KHÁM PHÁ DỮ LIỆU
df = pd.read_csv('TÊN_FILE.csv')

print("=== THÔNG TIN CƠ BẢN ===")
print(f"Kích thước: {df.shape}")
print(f"Các cột: {df.columns.tolist()}")
print(f"Kiểu dữ liệu:\n{df.dtypes}")
print(f"Dữ liệu thiếu:\n{df.isnull().sum()}")

print("\n=== MẪU DỮ LIỆU ===")
print(df.head())
print(df.describe())

# 2. XỬ LÝ DỮ LIỆU
# Xóa cột không cần thiết
if 'Unnamed: 0' in df.columns:
    df = df.drop('Unnamed: 0', axis=1)

# Xử lý dữ liệu thiếu
df = df.dropna() # hoặc df.fillna()

# 3. PHÂN TÍCH THỐNG KÊ MÔ TẢ
numeric_cols = df.select_dtypes(include=[np.number]).columns
categorical_cols = df.select_dtypes(include=['object']).columns

print("\n=== THỐNG KÊ CÁC CỘT SỐ ===")
for col in numeric_cols:
    print(f"\n{col}:")
    print(f"  Trung bình: {df[col].mean():.2f}")
    print(f"  Trung vị: {df[col].median():.2f}")
    print(f"  Độ lệch chuẩn: {df[col].std():.2f}")
    print(f"  Min-Max: {df[col].min()} - {df[col].max()}")

print("\n=== THỐNG KÊ CÁC CỘT PHÂN LOẠI ===")
for col in categorical_cols:
    print(f"\n{col}:")
    print(df[col].value_counts())

# 4. TRỰC QUAN HÓA
# Histogram cho các cột số
```

```

if len(numeric_cols) > 0:
    df[numeric_cols].hist(figsize=(15, 10), bins=20)
    plt.suptitle('Histogram các cột số')
    plt.tight_layout()
    plt.show()

# Bar chart cho các cột phân loại
for col in categorical_cols[:3]: # Chỉ vẽ 3 cột đầu
    plt.figure(figsize=(10, 6))
    df[col].value_counts().plot(kind='bar')
    plt.title(f'Biểu đồ cột - {col}')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

# Ma trận tương quan
if len(numeric_cols) > 1:
    plt.figure(figsize=(10, 8))
    sns.heatmap(df[numeric_cols].corr(), annot=True, cmap='coolwarm')
    plt.title('Ma trận tương quan')
    plt.show()

print("\n=== HOÀN THÀNH PHÂN TÍCH ===")

```

TEMPLATE 2: PHÂN PHỐI XÁC SUẤT

```

import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import binom, poisson, norm

# ===== PHÂN PHỐI NHỊ THỨC =====
print("PHÂN PHỐI NHỊ THỨC")
n, p = 10, 0.3 # Thay số theo đề

# Tính xác suất
for k in [0, 1, 2, 5]:
    prob = binom.pmf(k, n, p)
    print(f"P(X = {k}) = {prob:.6f}")

# Vẽ đồ thị
x = range(n+1)
probs = [binom.pmf(i, n, p) for i in x]
plt.figure(figsize=(10, 6))
plt.bar(x, probs, alpha=0.7)
plt.title(f'Phân phối nhị thức B({n}, {p})')
plt.xlabel('k')
plt.ylabel('P(X = k)')
plt.show()

# ===== PHÂN PHỐI POISSON =====
print("\nPHÂN PHỐI POISSON")
lam = 3 # Thay số theo đề

```

```

for k in [0, 1, 2, 5]:
    prob = poisson.pmf(k, lam)
    print(f"P(X = {k}) = {prob:.6f}")

x = range(15)
probs = [poisson.pmf(i, lam) for i in x]
plt.figure(figsize=(10, 6))
plt.bar(x, probs, alpha=0.7, color='orange')
plt.title(f'Phân phối Poisson( $\lambda={lam}$ )')
plt.xlabel('k')
plt.ylabel('P(X = k)')
plt.show()

# ===== PHÂN PHỐI CHUẨN =====
print("\nPHÂN PHỐI CHUẨN")
mu, sigma = 100, 15 # Thay số theo đề

# Tính xác suất
values = [90, 110, 120]
for val in values:
    prob = norm.cdf(val, mu, sigma)
    print(f"P(X < {val}) = {prob:.6f}")

# Vẽ đồ thị
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 1000)
y = norm.pdf(x, mu, sigma)
plt.figure(figsize=(10, 6))
plt.plot(x, y, 'b-', linewidth=2)
plt.title(f'Phân phối chuẩn N({mu}, {sigma}^2)')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.grid(True, alpha=0.3)
plt.show()

print("\n=== HOÀN THÀNH PHÂN TÍCH XÁC SUẤT ===")

```

TEMPLATE 3: TIPS DATA ANALYSIS

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Đọc dữ liệu
df = pd.read_csv('tips.csv')
if 'Unnamed: 0' in df.columns:
    df = df.drop('Unnamed: 0', axis=1)

print("=== THÔNG TIN DỮ LIỆU ===")
print(f"Kích thước: {df.shape}")
print(df.head())

```

```

# Tính tỷ lệ tip
df['tip_percentage'] = (df['tip'] / df['total_bill']) * 100

print("\n=== THỐNG KÊ CƠ BẢN ===")
print(f"Trung bình tip: ${df['tip'].mean():.2f}")
print(f"Tỷ lệ tip trung bình: {df['tip_percentage'].mean():.1f}%")
print(f"Số khách trung bình: {df['size'].mean():.1f}")

# Thống kê theo giới tính
print("\n=== THEO GIỚI TÍNH ===")
gender_stats = df.groupby('sex').agg({
    'tip': ['mean', 'count'],
    'tip_percentage': 'mean',
    'total_bill': 'mean'
}).round(2)
print(gender_stats)

# Thống kê theo ngày
print("\n=== THEO NGÀY ===")
day_stats = df.groupby('day').agg({
    'tip': ['sum', 'mean', 'count'],
    'total_bill': 'sum'
}).round(2)
print(day_stats)

# Biểu đồ
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Total bill histogram
axes[0, 0].hist(df['total_bill'], bins=20, alpha=0.7)
axes[0, 0].set_title('Histogram - Total Bill')

# Tip vs Total Bill scatter
axes[0, 1].scatter(df['total_bill'], df['tip'], alpha=0.6)
axes[0, 1].set_title('Tip vs Total Bill')
axes[0, 1].set_xlabel('Total Bill')
axes[0, 1].set_ylabel('Tip')

# Tips by day
day_tips = df.groupby('day')['tip'].sum()
axes[1, 0].bar(day_tips.index, day_tips.values)
axes[1, 0].set_title('Total Tips by Day')

# Tips by time
time_tips = df.groupby('time')['tip'].sum()
axes[1, 1].bar(time_tips.index, time_tips.values, color='orange')
axes[1, 1].set_title('Total Tips by Time')

plt.tight_layout()
plt.show()

print("\n=== HOÀN THÀNH PHÂN TÍCH TIPS ===")

```




CHECKLIST THI QUAN TRỌNG



TRƯỚC KHI BẮT ĐẦU:

1. **Import đầy đủ:** Copy toàn bộ import statements
2. **Đọc kỹ đề:** Gạch chân từ khóa quan trọng
3. **Kiểm tra file:** Tên file có đúng không?
4. **Kiểm tra cột:** `print(df.columns)` để biết tên cột chính xác



KHI LÀM BÀI:

1. **Thay tên đúng:** File name, column name, số liệu
2. **Chạy từng cell:** Đừng chạy tất cả cùng lúc
3. **Kiểm tra kết quả:** Có hợp lý không?
4. **Comment kết quả:** Viết nhận xét nếu đề yêu cầu



TRƯỚC KHI NỘP:

1. **Kiểm tra tên file notebook:** `stt_hoten_GK.ipynb`
2. **Thông tin đầu file:** MSV, họ tên, số máy
3. **Tất cả cell đã chạy:** Không có lỗi
4. **Đủ câu trả lời:** Tất cả câu hỏi đã làm



SỬA LỖI NHANH:

- **KeyError:** Sai tên cột → `print(df.columns)`
- **FileNotFoundError:** Sai tên file → kiểm tra chính tả
- **ValueError:** Sai kiểu dữ liệu → `print(df.dtypes)`
- **Import error:** Cài thư viện thiếu



CHIẾN THUẬT THI:

1. **Làm câu dễ trước:** Đọc file, thống kê cơ bản
2. **Copy template:** Sử dụng code mẫu, chỉ thay tên

3. Nhận xét đơn giản: "Phân phối lệch phải", "Tương quan dương"

4. Quản lý thời gian: 15-20p/câu lớn



MẸO HAY:

- **Ctrl+F** tìm nhanh code cần thiết trong cheat sheet này
- **Thay tên hàng loạt:** Ctrl+H trong VS Code
- **Backup code:** Copy vào file .txt để phòng khi mất
- **Test nhỏ:** Chạy với sample nhỏ trước khi chạy full



CÁC LOẠI BIỂU ĐỒ VÀ KHI NÀO DÙNG

Loại biểu đồ	Khi nào dùng	Code mẫu
Histogram	Phân bố của 1 biến liên tục	<code>plt.hist(df['col'], bins=20)</code>
Bar Chart	So sánh các nhóm riêng biệt	<code>df['col'].value_counts().plot(kind='bar')</code>
Pie Chart	Tỷ lệ phần trăm các thành phần	<code>plt.pie(counts.values, labels=counts.index)</code>
Boxplot	Phân bố và outliers	<code>plt.boxplot(df['col'])</code>
Scatter Plot	Tương quan 2 biến	<code>plt.scatter(df['x'], df['y'])</code>
Line Plot	Xu hướng theo thời gian	<code>plt.plot(df['time'], df['value'])</code>
Heatmap	Ma trận tương quan	<code>sns.heatmap(df.corr(), annot=True)</code>

🍀 CHÚC BẠN THI TỐT! NHỚ GIỮ BÌNH TĨNH VÀ LÀM TỪNG BƯỚC! 🍀 📝
COPY CHEAT SHEET NÀY VÀO USB ĐỂ DÙNG KHI CẦN! 📝